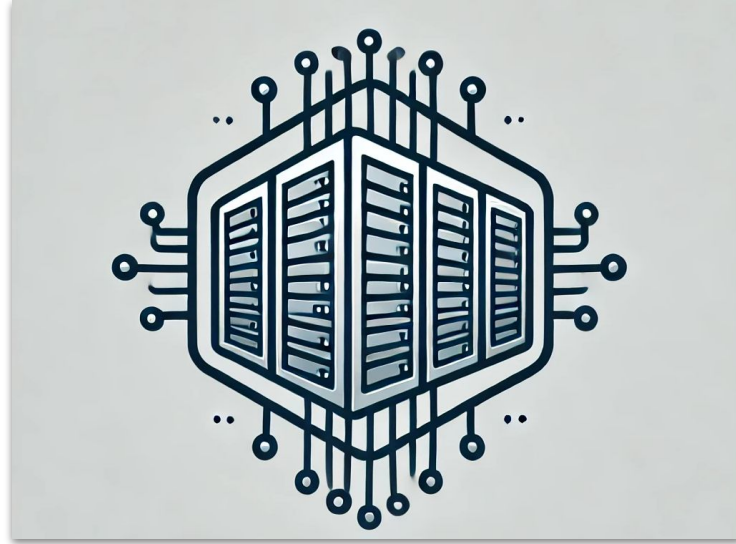


CSC 746: High Performance Computing

Parallel Perf Measures, SMP Programming

24 Sep 2026



What's up today

Last time:

- CP2 discussion
- Shared-memory parallelism

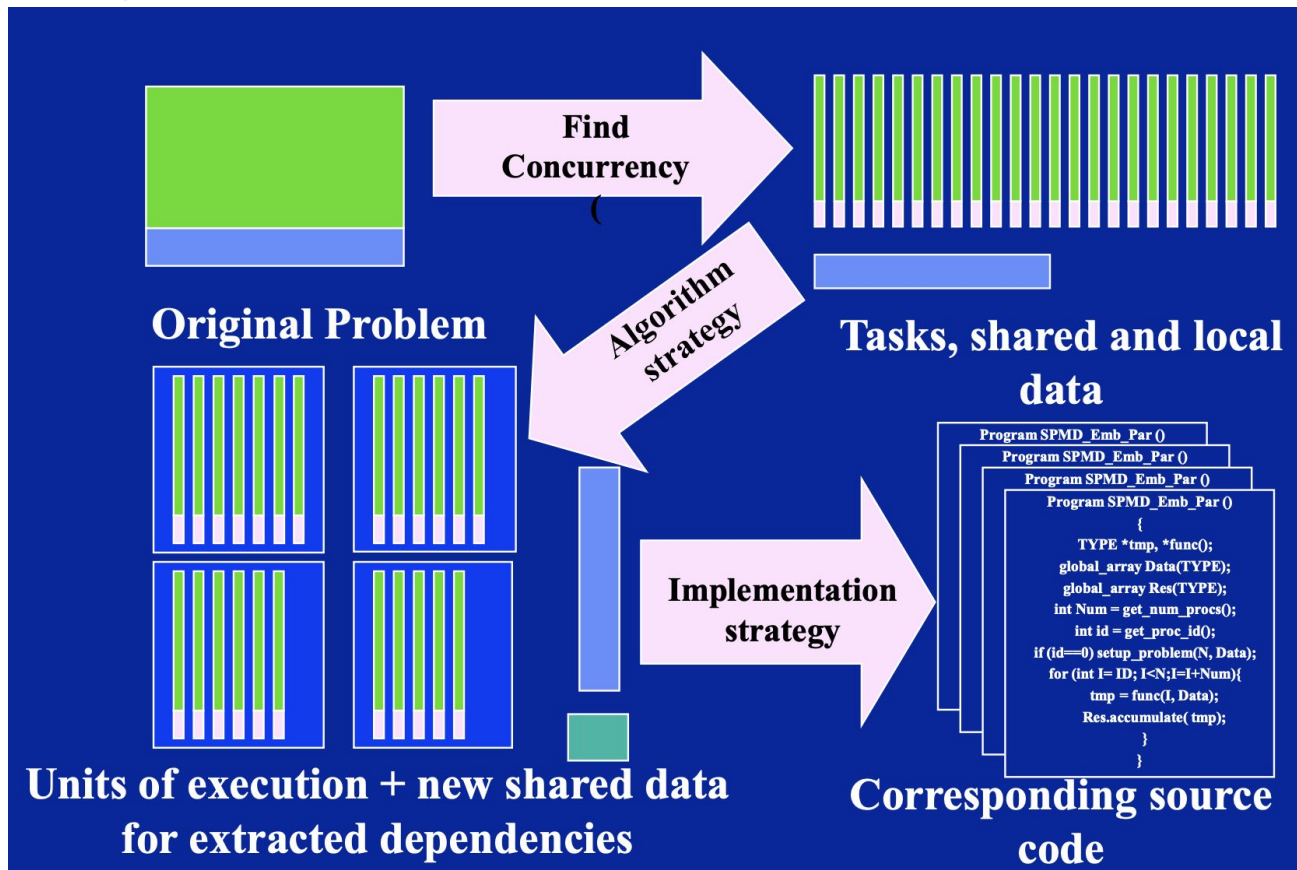
Today:

- Parallel performance measures
- Shared-memory parallel programming
 - POSIX threads
 - OpenMP
- Hello CP#3
- (Everyone) in-class presentation/idea pitch on Tue 9/29/2026



The Process of Going from Serial To Parallel

- Identifying parallelism
- Overhead balance
- Load balance
- Communication
- Locality
- Synchronization and coordination



Part 2 of CSC 746 - Shared memory parallelism

Considerations:

- Identify parallelism
- Consider also overhead balance
- Load balance
- Communication
- Locality
- Synchronization and coordination

As we:

- Write parallel code
- Measure performance, scalability
- Optimizations to work around performance limits

Parallel Performance Measures

Terminology in Parallel Computing

N - the problem size

- A measure of the amount of work to be done
- Independent of algorithm complexity: $O(N)$, $O(\log(N))$, $O(N^2)$, etc.

P - parallelism

- The number of parallel “processes”
- Independent of, but related to, how the problem N is divided up among the P workers

Rank

- From MPI, refers to one of the P workers. “A rank”, “Rank 0”, or “ P ranks”
- Logically independent of how implemented: can have multiple ranks actually map to a single hardware thread, this is an O/S implementation issue

Parallel Performance Measures

Image source: wikipedia.com

Speedup* - how much faster a parallel code performs compared to a serial version

Scalability* - is speedup linear, sublinear, superlinear with increasing P ?

Load balance - degree of variance in execution time across P ranks



* Caveat: for the present, we are oversimplifying some key concepts and terms. These terms will unfold into other terms and concepts as we go on.

Speedup: How much faster is parallel vs. serial?

Speedup is a *ratio* of a parallel program's speed to a sequential program's speed

For a problem size n and p parallel ranks:

$T^*(n)$ is the time for the best serial algorithm on a problem of size n

$$S(n, p) = \frac{T^*(n)}{T(n, p)}$$

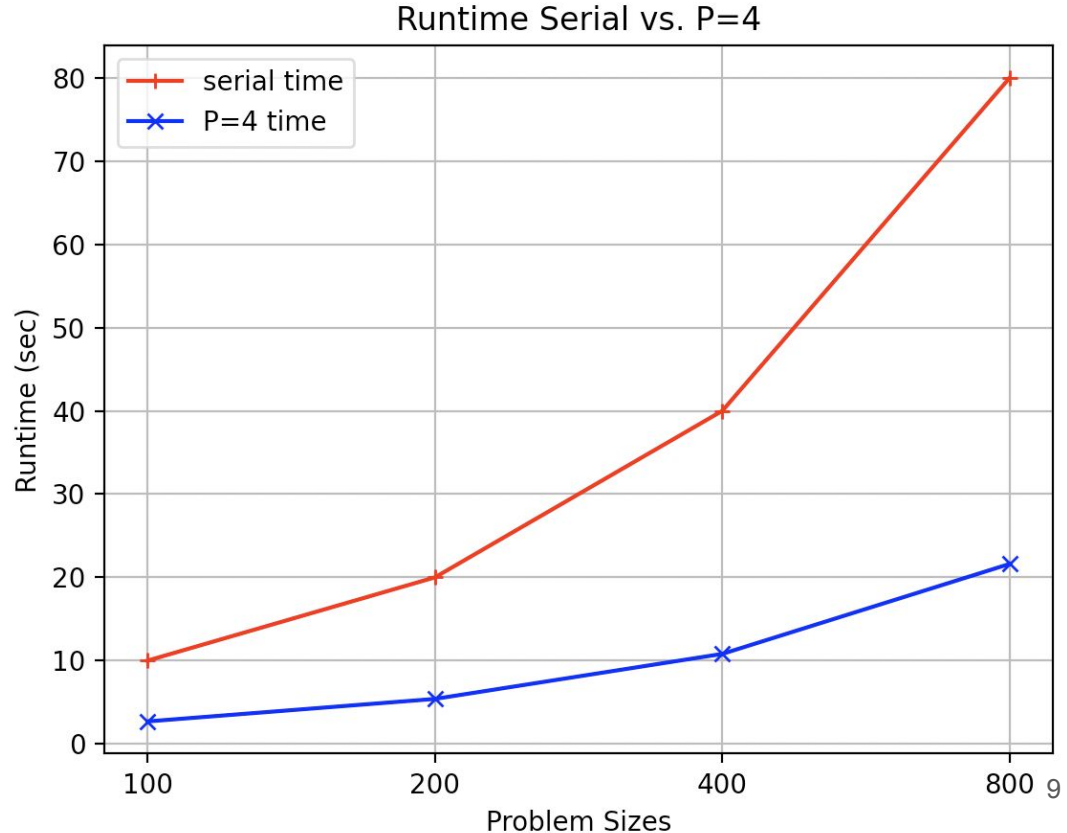
$T(n, p)$ is the time it takes to run the program on a problem of size n using p parallel ranks

Useful reference:
<https://hpc-wiki.info/hpc/Scaling>

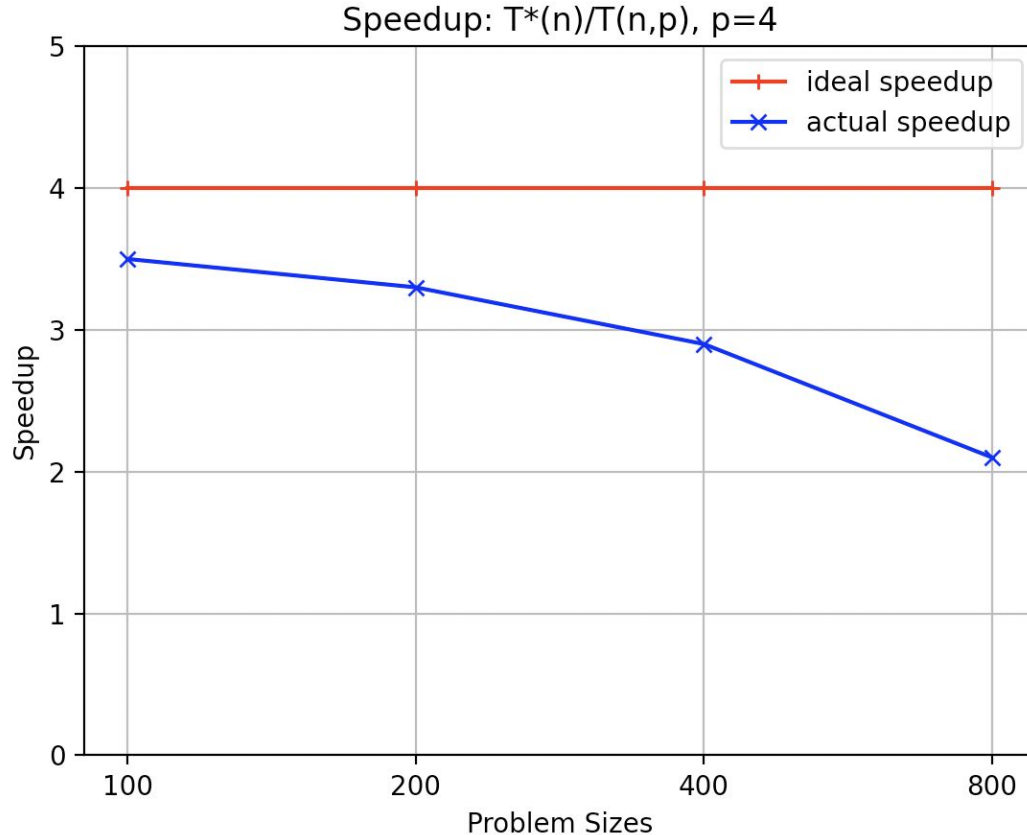
Speedup “Warm-up”: Plot of Runtime

Two codes: serial, and OpenMP p=4
Varying problem size N

	A	B	C
1	Problem size	serial	p=4 time
2	100	10	2.8571429
3	200	20	6.0606061
4	400	40	13.793103
5	800	80	38.095238
6			



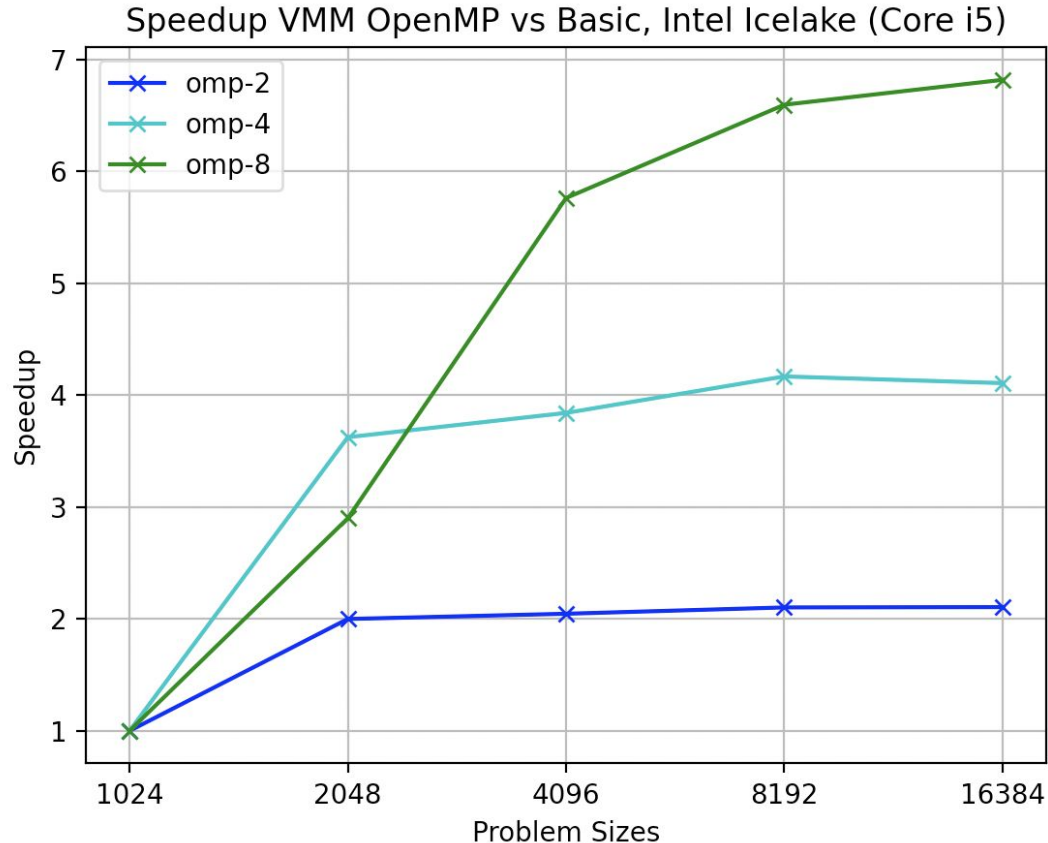
Speedup Plot Example 1



	A	B	C	D
				speedup $T^*(n)/T(n,p)$
1	Problem size	serial	p=4 time	
2	100	10	2.8571429	3.5
3	200	20	6.0606061	3.3
4	400	40	13.793103	2.9
5	800	80	38.095238	2.1

$$S(n, p) = \frac{T^*(n)}{T(n, p)}$$

Speedup Plot Example 2



Intel Icelake 2.00 GHz
4 cores
L2 Cache: 512 KB per core
L3 Cache: 6 MB
Hyperthreading: Enabled

$$S(n, p) = \frac{T^*(n)}{T(n, p)}$$

Discussion of results: part of
your CP3 assignment.

This result at N=1024 is correct: the runtimes for serial, omp-2, omp-4, omp-4 are all about the same so the speedup times (time for serial divided by time for parallel) are all about 1.0 at N=1024.

What Do Speedup Plots Tell Us?

For a given problem size N , the *Speedup* metric shows how much faster are we able to solve the problem using P ranks rather than using a serial implementation

Why is the maximum possible $\text{Speedup} = P$??

What does it mean if $\text{Speedup} > P$??

What does it mean if $\text{Speedup} < P$??



Strong Scaling

Refers to running a problem of size N on P ranks rather than on 1 rank:

- Problem size is constant = N
- Increase parallelism P

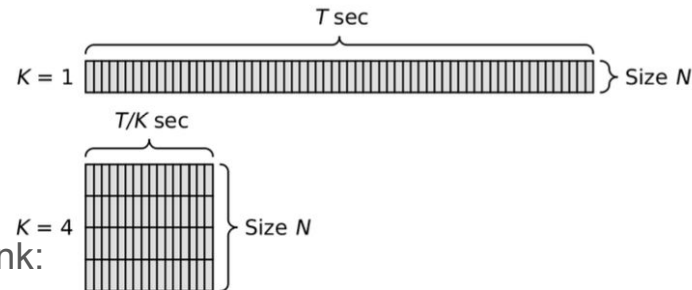


Image credit: Kaminsky, 2015

Ideally, each of the P ranks works on N/P of the problem (equal work)

Ideally, the serial runtime, $T^*(N)$, is reduced by a factor of $1/P$ when there is *ideal* strong scaling

- When $P=2$, the problem completes in $\frac{1}{2}$ the time of $T^*(N)$
- When $P=8$, the problem completes in $\frac{1}{8}$ the time of $T^*(N)$, etc.

Realistically, things don't often work out that way, and $T^*(N)$ is reduced by $1/(P+F)$

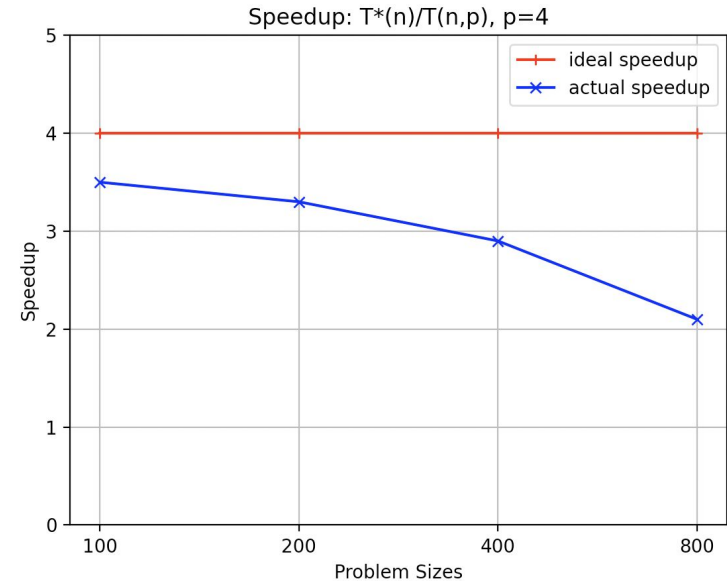
What is F ?

Amdahl's Law

For any given program, only a portion of it can be run in parallel, while the remainder must be run in serial.

As a result, there is a limit to the achievable speedup in strong scaling

G. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. *Proceedings of the AFIPS Spring Joint Computer Conference*, 1967, pages 483–485.



Amdahl's Law, ctd

Let:

f be the portion of the program that must be serial (f in $[0..1]$)

$(1-f)$ is the portion of the program that can run in parallel

The limit to scaling for a given n and p is then:

Smaller f : closer to ideal scaling
Larger f : further from ideal scaling

$$S(n, p) = \frac{1}{f + (1 - f)/p}$$

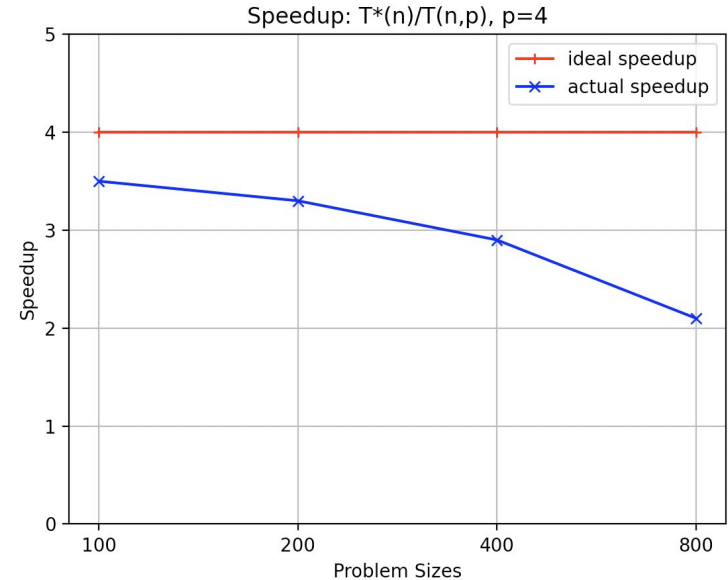
Amdahl's Law, ctd.

The effect becomes amplified with increasing p

More time spent doing serial work: $f \cdot P$

Less time spent doing parallel work: $(1-f) \cdot P$

$$S(n, p) = \frac{1}{f + (1 - f)/p}$$

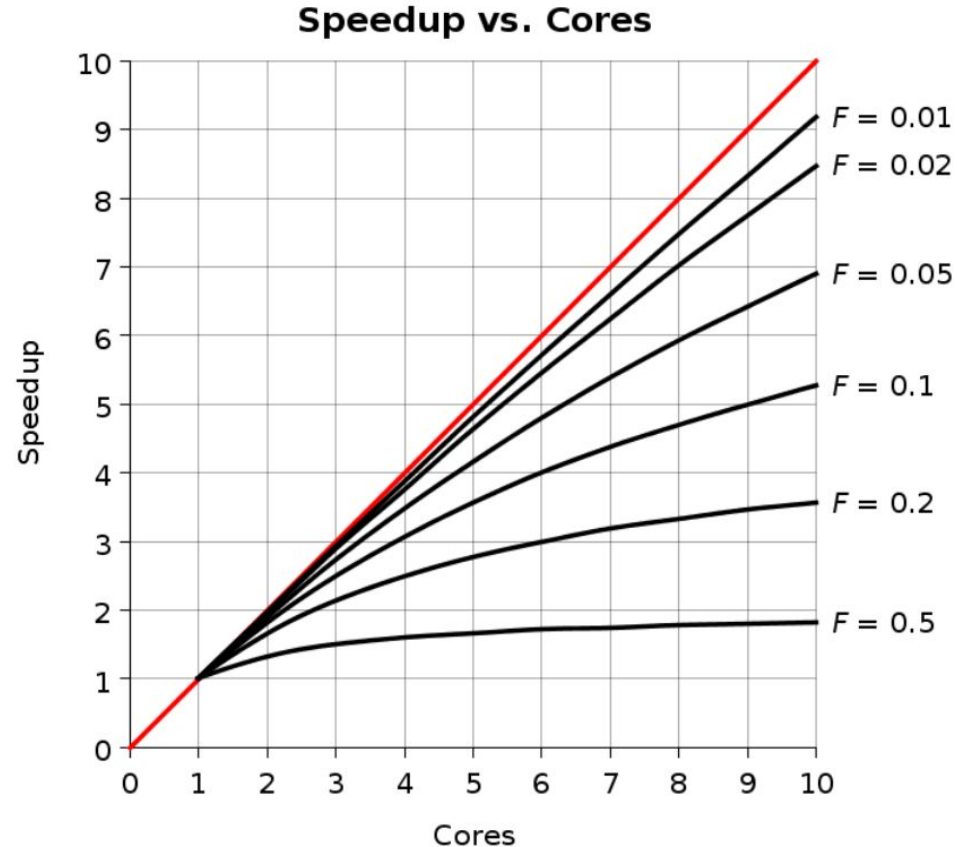


Amdahl's Law, ctd.

Increasing values of f increasingly limit scalability

In the limit as $f \rightarrow 1.0$, $S(n,p) = 1/f$

$$S(n,p) = \frac{1}{f + (1-f)/p}$$



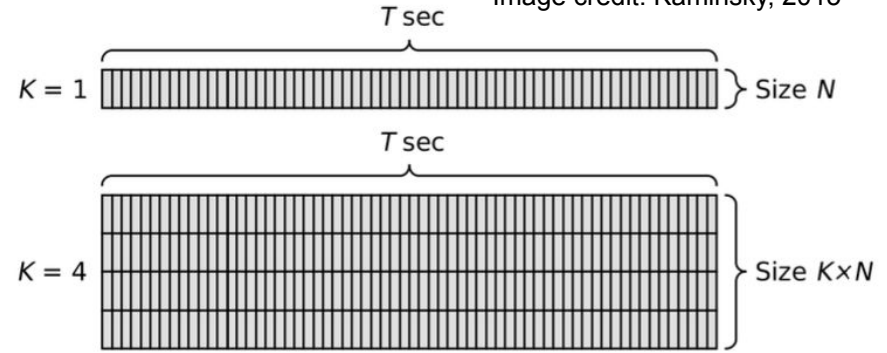
Weak Scaling

Recall: strong scaling holds N constant and increases P to reduce the time to solution

Weak scaling: as P increases, so does N ; perfect scaling occurs when $T(1) == T(P)$

A key idea for using larger machines to tackle larger problems:

- Higher resolution climate grids
- More complex systems in molecular dynamics models



J. Gustafson. Reevaluating Amdahl's law.
Communications of the ACM, 31(5):532–533, May 1988.

Gustafson's Law

J. Gustafson. Reevaluating Amdahl's law.
Communications of the ACM, 31(5):532–533, May
1988.

Recall:

- Problem size = N
- Number of processors = P

A performance model focusing on scalability of parallel computing as N increases

Key observation:

- When increasing P , can also increase N to achieve greater parallel efficiency

Gustafson's, ctd.

Scalability with increased P

- Rather than hold N constant and increase P, we also increase N

Constant execution time

- If time required on size N for $P=1$ is t , then
- Time required for size $N \cdot P$ on P processors is also t

Parallel workload

- Proportion of the workload that can be parallelized is what defines performance as P increases

$$S(N) = N - \alpha(N - 1)$$

Gustafson's, ctd.

Where:

- $S(N)$ is the speedup of the system with N processors.
- N is the number of processors.
- α is the fraction of the workload that is serial (non-parallelizable).

Amdahl: limits speedups by focusing on the serial portion of the code

- As P (here, N) increases, the serial portion dominates

Gustafson: as problems scale in size, the parallel portion grows significantly

- As P (here, N) increases, the parallel portion dominates

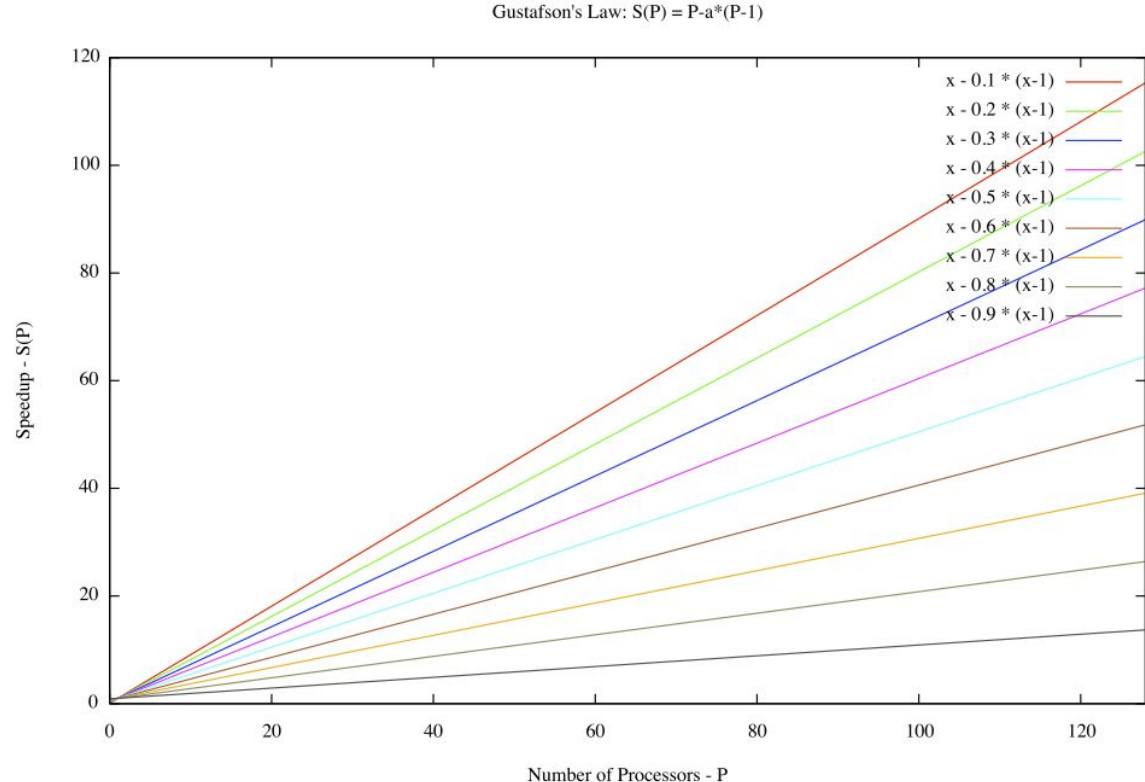
Gustafson's Law, ctd.

If a is the serial portion of the code (0.0...1.0)

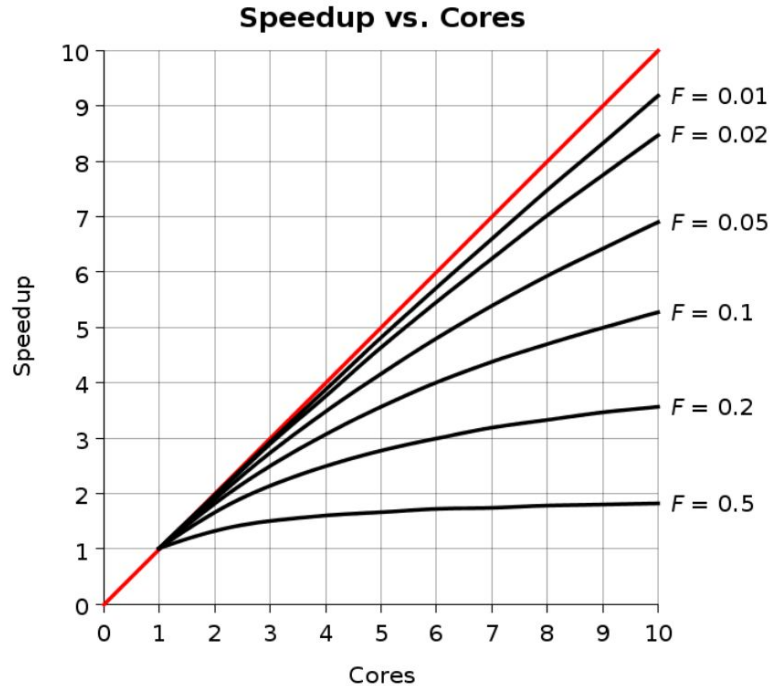
When $a \rightarrow 0$, then the potential speedup $\rightarrow P$

As $a \rightarrow 1$, then the potential speedup $\rightarrow 0$

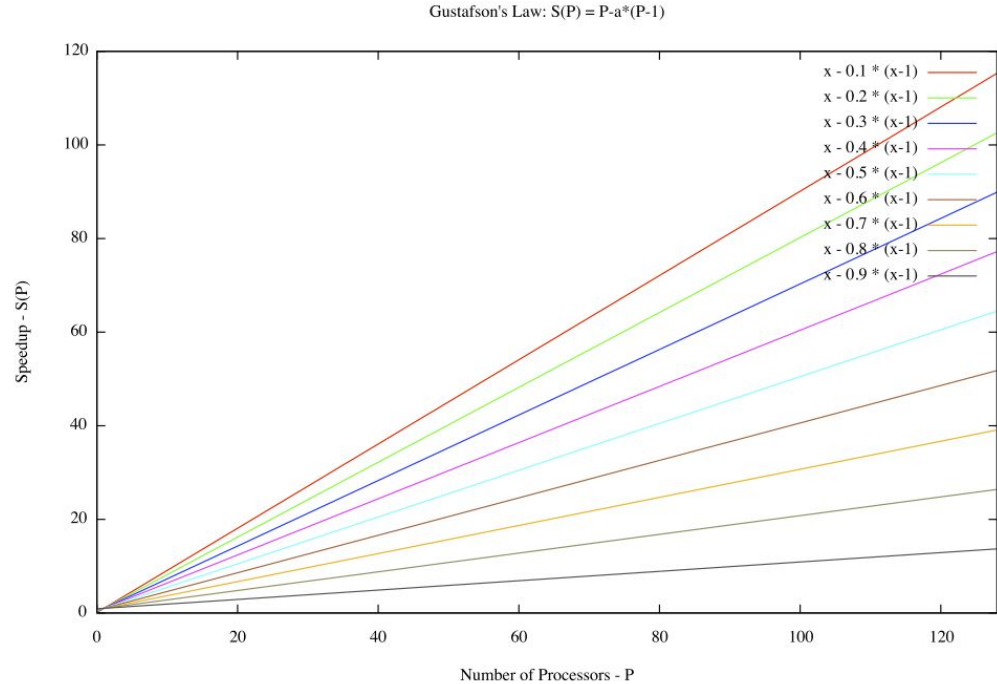
Not a surprise...



Amdahl's vs. Gustafson's



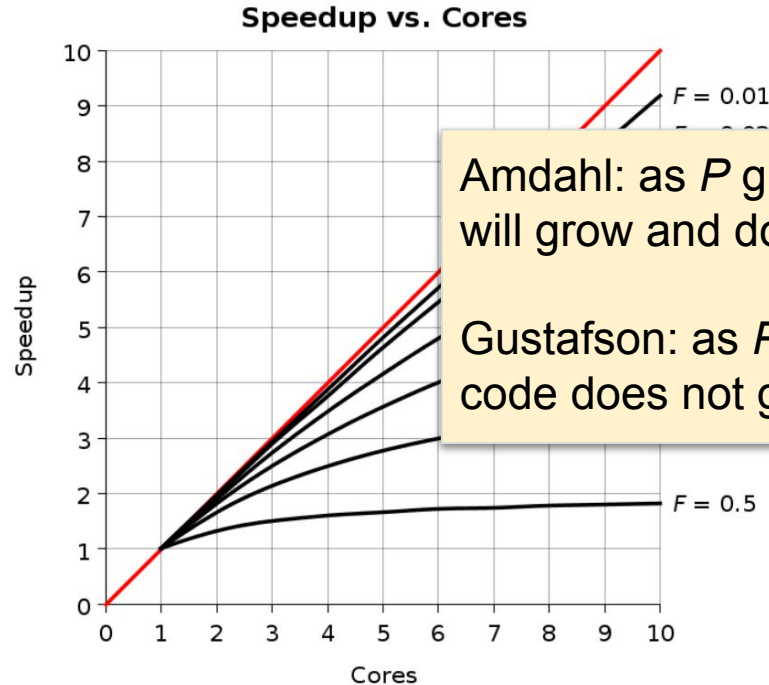
Amdahl: The mere presence of serial code limits scalability (strong scaling)



Gustafson: the presence of serial code reduces but doesn't limit scalability (weak scaling)

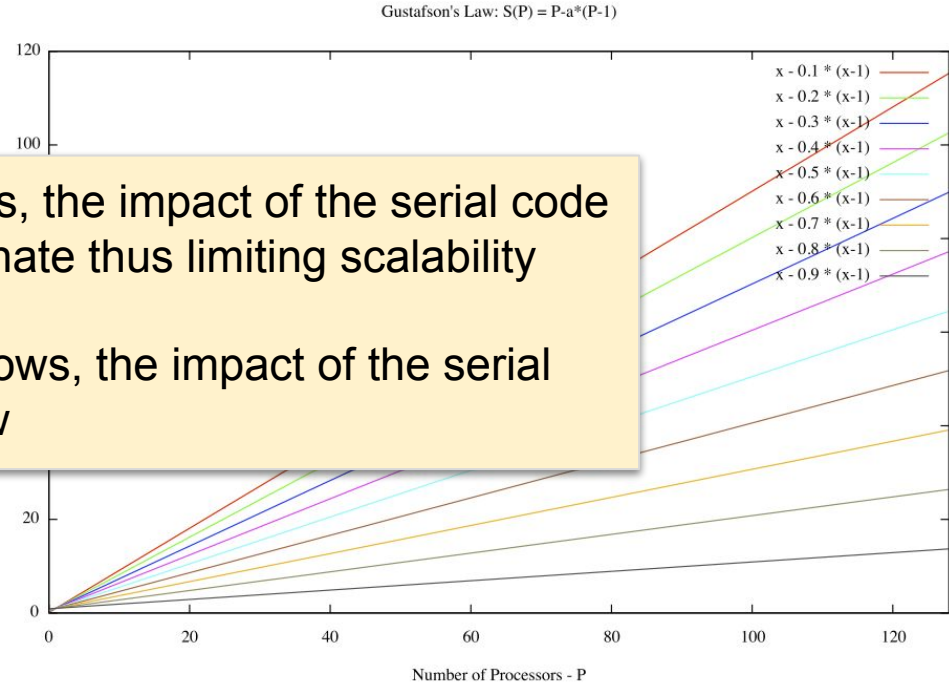


Amdahl's vs. Gustafson's



Amdahl: as P grows, the impact of the serial code will grow and dominate thus limiting scalability

Gustafson: as P grows, the impact of the serial code does not grow



Amdahl: The mere presence of serial code limits scalability (strong scaling)

Gustafson: the presence of serial code reduces but doesn't limit scalability (weak scaling)

Computational Rate: Weak Scaling Performance Measure

Rather than think about speedup, instead think about *computational rate*

Ratio of problem size to running time

$$R(n, p) = \frac{n}{T(n, p)}$$

$R(n, p)$ is the rate at which computations are performed for a given problem size n and parallelism level p

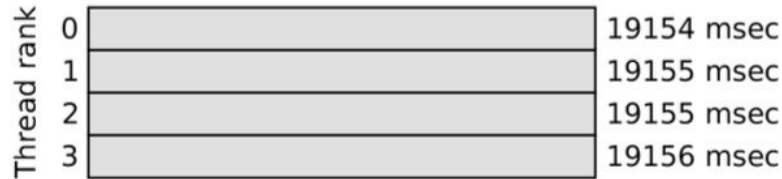
$T(n, p)$ is the time required to do the computation for a given problem size n and parallelism level p

Parallel Performance Metric: Load Balance

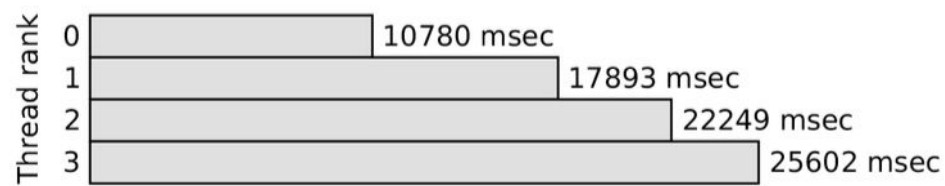
Key problem in parallel computing: dividing up the N evenly across the P

Desired outcome: each P does the same amount of work

Examples:



Good load balance: each thread executes in about the same amount of time.



Load imbalance: over 2.5x difference between fastest and slowest thread

Sources of Load Imbalance

Uneven work distribution across the set of p

Data dependency: for some problems, the amount of work required varies as a function of the input data

Systemic fluctuations: some of the p 's may be delayed due to system issues

(These are all active research areas)

Quantifying Load Imbalance

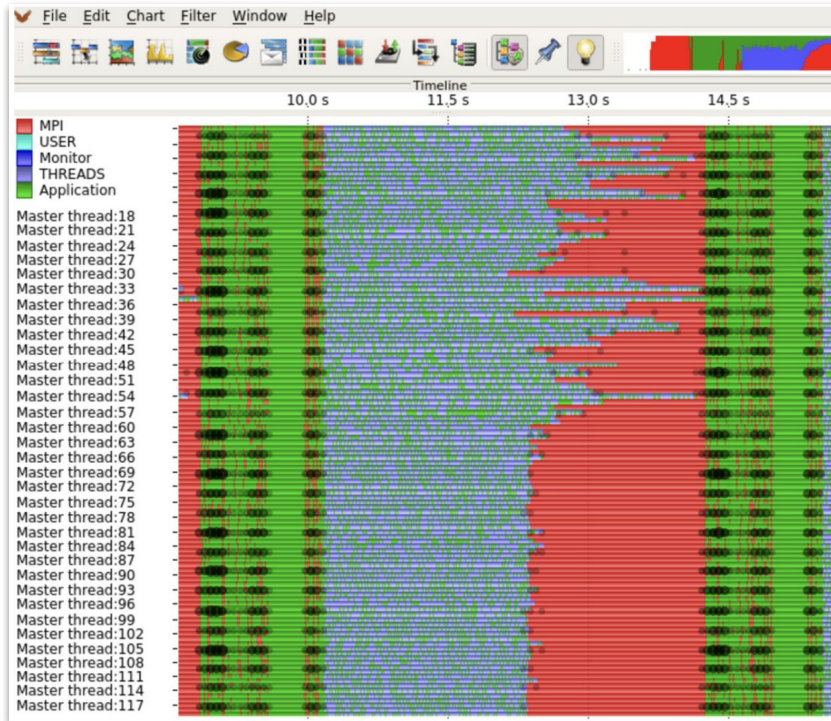
Straightforward approach:

- Measure elapsed time for all P 's
- Compute a statistical measure, like standard deviation:

$$\sigma = \sqrt{\sum (T_p - \mu)^2}$$

Many other ways (this is a big research area)

Visualizing Load Imbalance



Performance profiling: green denotes work done by the solver, gray shows chemistry computations, red shows time spent waiting.

Image credit: Zirwes et. al, 2018, Optimizing Load Balancing of Reacting Flow Solvers in OpenFOAM for High Performance Computing.



Shared-memory parallel programming

Shared-memory Programming



fork() system call - Berkeley Unix, late 1980s-early 1990s

- Creates a child process of the calling (master process)
- Same address space as the caller
- Communicates by reading/writing shared memory

This is how all early OSs, web browsers, persistent network services, etc. were built “back in the day”

Pro: straightforward coding model

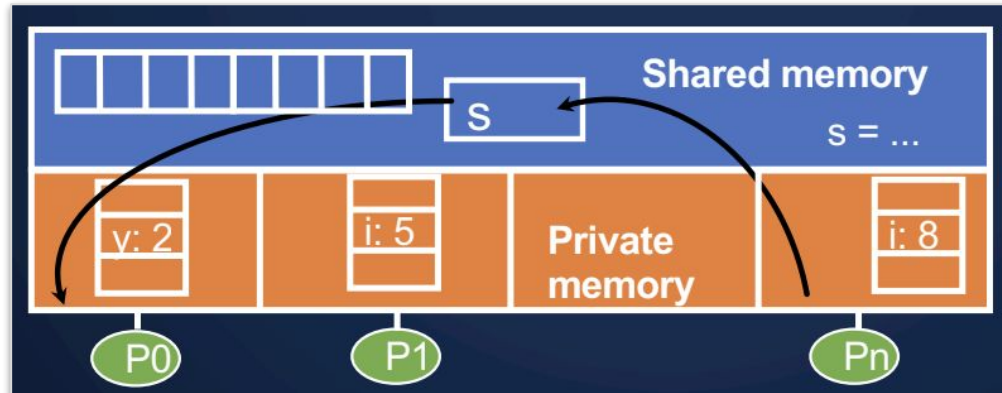
Cons:

- Creating a process is “heavyweight” and expensive (not good for HPC), various limits to scalability to high process count, large numbers of open files, etc.
- Explicit, manual methods for creating “shared memory segments”

Shared Memory Programming

Program is a collection of *threads of control*

- Can be created dynamically, mid-execution in some languages
 - Each thread has a set of private variables (thread-local storage)
 - Also a set of shared variables, e.g., static, global heap (visible to all threads)
- Threads communicate implicitly by writing and reading shared variables
- Threads coordinate by synchronizing on shared variables



Threads - Popularized in Solaris OS (Sun Microsystems)

Key ideas/objectives:

Avoid expensive startup cost associated with launching a new process

Keep a pool of threads in the kernel, then map them to a “lightweight process” at runtime as needed by user code

Specialized thread library in Solaris (early 1990s)

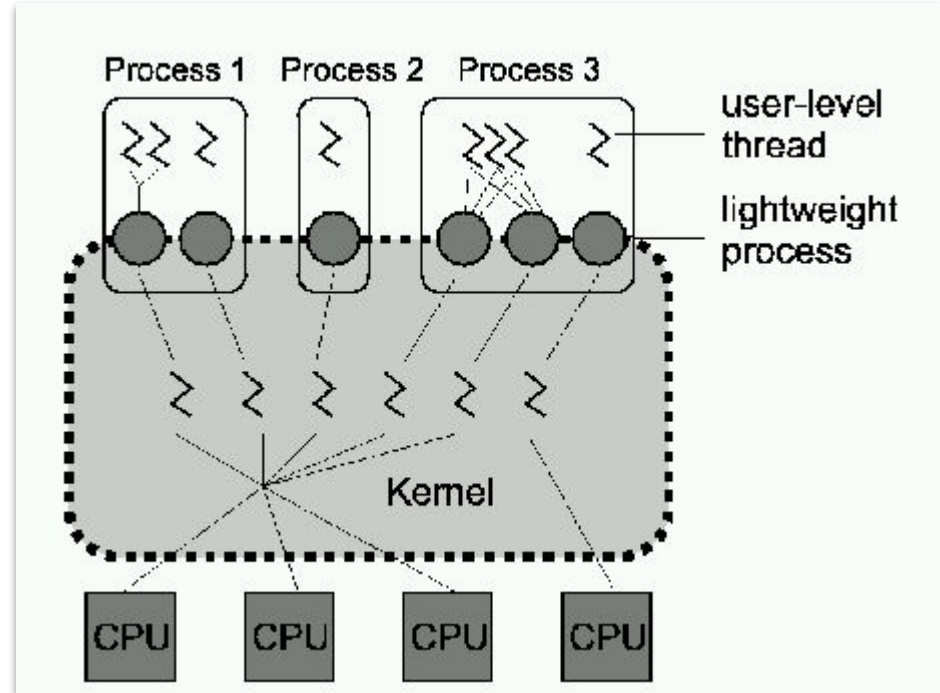


Image Credit: linuxjournal.com

Copyright © 2026, E. Wes Bethel

33

POSIX Threads (pthreads) (1995ish-present)

A library providing methods for applications to:

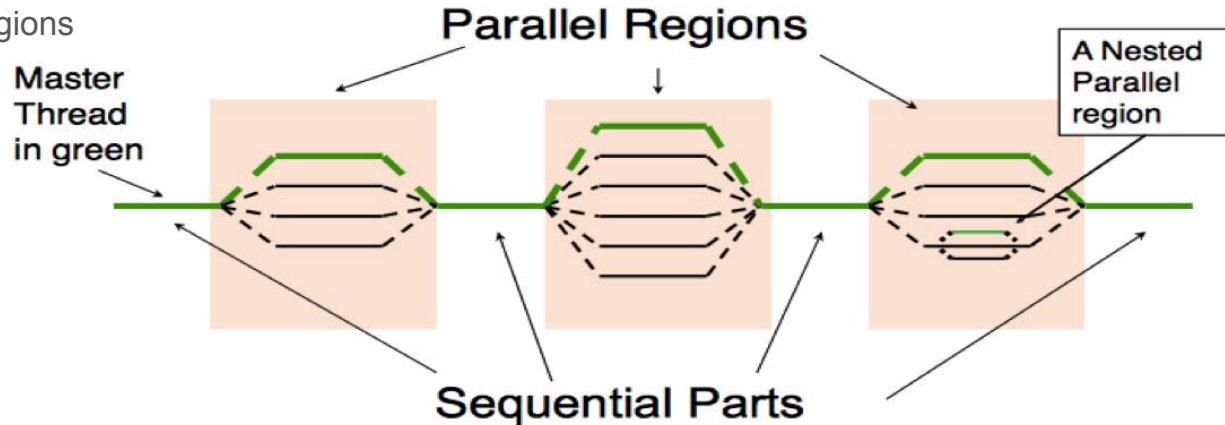
- Create, join threads
- Synchronize access to critical regions
- Condition variables

Broad industry support, many vendor implementations

POSIX standard: Portable Operating System Interface

Write code once, it runs on all POSIX-compliant OS's (on CPUs, not GPUs)

pthreads is the POSIX threading interface



“Forking” POSIX threads

Signature:

```
int pthread_create(pthread_t *,
                  const pthread_attr_t *,
                  void * (*)(void *),
                  void *);
```

Example call:

```
errcode = pthread_create(&thread_id; &thread_attribute
                        &thread_func; &fun_arg);
```

- **thread_id** is a handle, used to join, halt, etc.
- **thread_attribute** : various attributes like stack size, priority. NULL gives default attribs
- **thread_func**: the function to be run by the new thread
- **func_arg**: a pointer to arg/args passed to thread_func
- **errorcode**: will be set to nonzero if the create operation fails

Pthreads Example

L17: `pthread_create()`
creates a thread and
starts it executing

L5: Each thread runs
the function `hello()`

L21: `pthread_join()`
waits for a thread to
finish

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 /* each thread executes this function */
5 void *hello(void *foo) {
6     printf(" Hello world! \n");
7     return NULL;
8 }
9
10 #define NTHREADS 4
11 int main() {
12     pthread_t threads[NTHREADS];
13     int tn;
14
15     /* first, create the threads and start them running */
16     for (tn=0;tn<NTHREADS;tn++)
17         pthread_create(&threads[tn], NULL, hello, NULL);
18
19     /* then wait for the threads to finish */
20     for (tn=0;tn<NTHREADS;tn++)
21         pthread_join(threads[tn], NULL);
22
23     return 0;
24 }
25
```



Pthreads w/args

pthread_create(..., function, ...)

- Maps user function() to a thread and launches it
- Can pass arguments

function(args) {do work}

- The function runs in a separate thread

pthread_join(...)

- Waits for forked threads to finish
- “Fork-join” model



```
#define NUM_THREADS 5
```

Image Credit: wikipedia.org

```
void *perform_work(void *arguments){
    int index = *((int *)arguments);
    int sleep_time = 1 + rand() % NUM_THREADS;
    printf("THREAD %d: Started.\n", index);
    printf("THREAD %d: Will be sleeping for %d seconds.\n", index, sleep_time);
    sleep(sleep_time);
    printf("THREAD %d: Ended.\n", index);
}

int main(void) {
    pthread_t threads[NUM_THREADS];
    int thread_args[NUM_THREADS];
    int i;
    int result_code;

    //create all threads one by one
    for (i = 0; i < NUM_THREADS; i++) {
        printf("IN MAIN: Creating thread %d.\n", i);
        thread_args[i] = i;
        result_code = pthread_create(&threads[i], NULL, perform_work, &thread_args[i]);
        assert(!result_code);
    }

    printf("IN MAIN: All threads are created.\n");

    //wait for each thread to complete
    for (i = 0; i < NUM_THREADS; i++) {
        result_code = pthread_join(threads[i], NULL);
        assert(!result_code);
        printf("IN MAIN: Thread %d has ended.\n", i);
    }

    printf("MAIN program has ended.\n");
    return 0;
}
```

Work Decomposition Problem

Assume we have an array $A[N]$

We want to divide the work of N items across P threads

What might that code look like?

- Step 1: Inside a for loop, set limits based on loop iteration, make function call
- Step 2: modify that model to use pthreads

Work Decomp

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <math.h>
4
5 typedef struct
6 {
7     int thread_num; /* which thread am I? */
8     int start, end; /* what is my subset of the 1D d
9     int return_val; /* nice to have return values */
10    double *A; /* the data to work with */
11 } ThreadArgs;
12
13 /* each thread executes this function */
14 void *cos_subset(void *args) {
15     ThreadArgs *ta = (ThreadArgs *)args;
16     double *my_A = ta->A;
17
18     printf(" thread %d: inside cos_subset, start, end
19     ->start, ta->end);
20
21     /* do some work */
22     for (int i=ta->start; i<ta->end; i++)
23         my_A[i] = cos(my_A[i]);
24
25     ta->return_val = ta->end - ta->start + 1; /* rep
26
27     return NULL;
28 }
```

```
28
29 #define N (1<<20)
30 #define NTHREADS 4
31 int main() {
32
33     pthread_t threads[NTHREADS];
34     ThreadArgs ta[NTHREADS];
35     int tn;
36     double *A = (double *)malloc(sizeof(double)*N);
37
38     /* initialize A -- this is serial */
39     double x=0.0, dx = 2.0*M_PI / (double)N;
40     for (int i=0; i<N; i++, x+=dx)
41         A[i] = x;
42
43     /* launch threads to compute A[i] = cos(A[i]); */
44     for (tn=0; tn<NTHREADS; tn++)
45     {
46         /* find start/end limits for each thread */
47         ta[tn].start = tn*(N / NTHREADS);
48         ta[tn].end = ta[tn].start + (N / NTHREADS - 1); /* potential bugs */
49         ta[tn].thread_num = tn;
50         ta[tn].A = A;
51         ta[tn].return_val = -1; /* init to something */
52         pthread_create(&threads[tn], NULL, cos_subset, (void *)(&ta+tn));
53     }
54
55     /* then wait for the threads to finish */
56     for (tn=0; tn<NTHREADS; tn++)
57     {
58         pthread_join(threads[tn], NULL);
59         printf(" Thread %d processed %d items \n", ta[tn].thread_num, ta[tn].return_val);
60     }
61
62     return 0;
63 }
```



Loop Level Parallelism in Pthreads

Concept:

Fine-grained vs.
coarse-grained parallelism

Many scientific applications have parallelism in loops

With threads:

```
for i = 0, N
  for j = 0, N
    pthread_create(update_cell[i][j], ...
                  my_array[i][j]);
```

But the overhead of thread creation is non-trivial

- update_cell() should have a significant amount of work
- 1/p'th of total work if possible

Data Race Example

```
static int s = 0;
```

Thread 1

```
for i = 0, n/2-1  
  s = s + f(A[i])
```

Thread 2

```
for i = n/2, n-1  
  s = s + f(A[i])
```

The problem: a *race condition* on the variable *s*

What is a *race condition*?

- When two or more threads try to write the same memory location
- Whoever writes last wins

A *race condition* occurs when:

- Two or more processes or threads attempt to access the same variable and at least one of them is doing a write
- The accesses are concurrent, but not synchronized, so they may happen in any order, producing unpredictable results



Basic Types of Synchronization: Mutexes

Mutexes – mutual exclusion, aka “locks”

- Threads are working mostly independently
 - Need access to a common data structure
- ```
lock *l = alloc_and_init(); /* shared */
acquire(l);
 Safely access data here
release(l);
```
- Locks only affect processors using them
    - A “rogue” thread can still access data w/o acquiring/releasing the mutex
    - All threads need to “play nicely” and use mutexes on critical data
  - Java, C++ and others have lexically scoped synchronization
  - Semaphores generalize locks to allow  $P$  threads simultaneous access: good for limited resources

# Mutexes in POSIX Threads

- To create a mutex:

```
#include <pthread.h>
pthread_mutex_t amutex = PTHREAD_MUTEX_INITIALIZER;
// or pthread_mutex_init(&amutex, NULL);
```

- To use it:

```
int pthread_mutex_lock(amutex);
int pthread_mutex_unlock(amutex);
```

- To deallocate a mutex

```
int pthread_mutex_destroy(pthread_mutex_t *mutex);
```

- Multiple mutexes may be held, but can lead to problems:

```
thread1
lock (a)
lock (b)
```

```
thread2
lock (b)
lock (a)
```



- Deadlock results if both threads acquire one of their locks, so that neither can acquire the second

# Summary of Programming with POSIX Threads

POSIX Threads are based on OS features

- Can be used from multiple languages
- Familiar language for most of program
- Ability to share data is convenient

Pitfalls

- Overhead of thread creation is high: once per inner loop iter is too much
- Data race bugs are hard to find: intermittent
- Deadlocks are easier to find, but can also be intermittent
- Coding overhead of implementing data parallelism

Researchers look at transactional memory models as an alternative (sync)

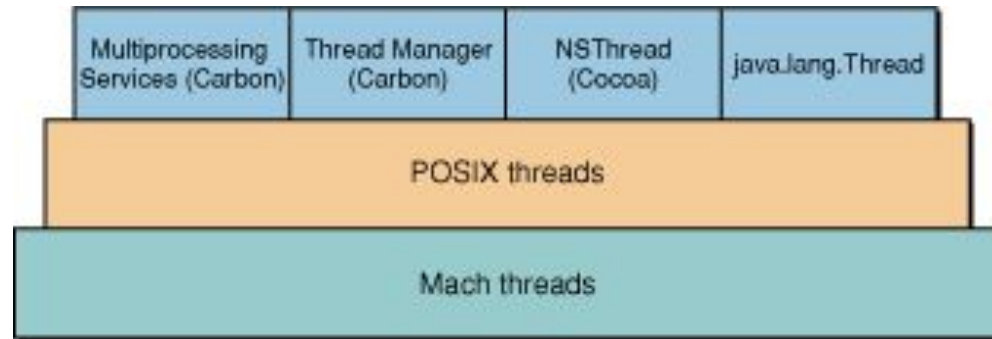
# Pthreads: an important part of the computing landscape

Pthreads are the “backbone” for many different aspects of modern multiprocessing

Not used so much in HPC applications

“task parallelism” as opposed to “data parallelism”

Likely a “dead end” if want platform portability across both CPUs and GPUs



Threading packages in MacOSX  
Image source: Apple Developer Connection

# Implementing shared-memory parallel code with OpenMP

# The OpenMP Standard

A set of compiler directives and library routines for writing multi-threaded, shared-memory parallel code

Includes environment variables that influence run-time behavior

Greatly simplifies writing multi-threaded programs in Fortran, C, and C++ for execution on shared-memory platforms

An easy way to get started with parallel coding

OpenMP is managed by a technology consortium (Intel, IBM, Cray, HP, Fujitsu, Red Hat, AMD, Cray, ...)



Supported by virtually all modern C, C++, Fortran compilers and OS's

# Programmer's View of OpenMP

OpenMP is a portable, threaded, shared-memory programming *specification* with “light” syntax

- Combination of `#pragma`'s and function calls
- Requires compiler support

OpenMP will:

- Enable a programmer to specify *parallel regions of code* that execute in parallel
- Hides stack management
- Provides synchronization constructs

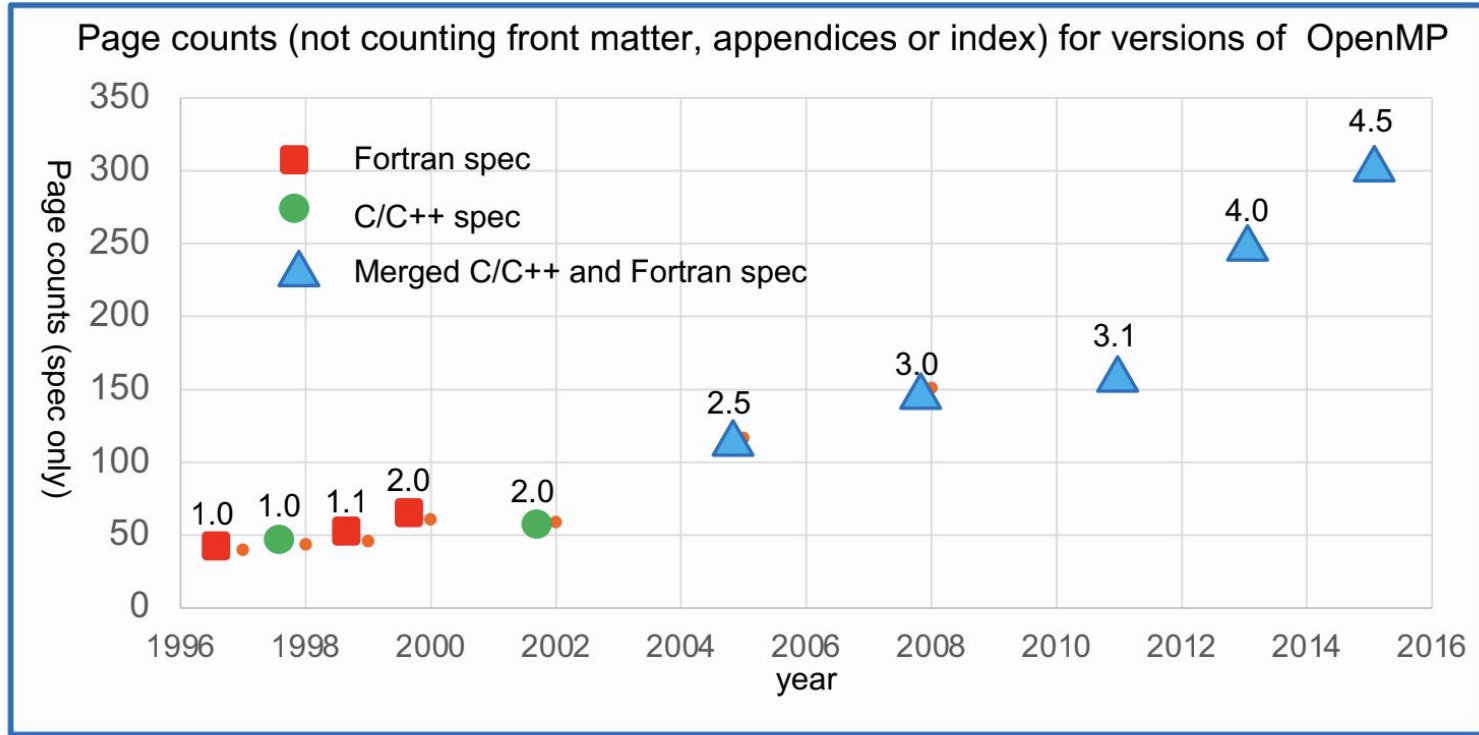
OpenMP will *not*:

- Parallelize automatically
- Guarantee speedup
- Provide freedom from data race conditions





# Growth of Complexity in OpenMP Over Time



OpenMP 5.0 (November 2018) is actually **666 pages**.

# OpenMP Example: Loop Parallelism

Specify where parallelism happens: the *#pragma*

*#pragma omp parallel for* will produce parallel code where each thread will execute a portion of that loop

E.g., if there are two threads, then thread 0 executes  $i=0..50000$  and thread 1 executes  $i=50001..100000$

All other portions of the code are serial

```
int main(int argc, char **argv)
{
 int a[100000];

 #pragma omp parallel for
 for (int i = 0; i < 100000; i++) {
 a[i] = 2 * i;
 }

 return 0;
}
```

# OpenMP Hello World

```
1 #include <omp.h>
2 #include <stdio.h>
3
4 int main ()
5 {
6 #pragma omp parallel
7 {
8 // ID of the thread in the current team
9 int thread_id = omp_get_thread_num();
10 // Number of threads in the current team
11 int nthreads = omp_get_num_threads();
12
13 printf("I'm thread %d out of %d threads.\n", thread_id, nthreads);
14 }
15 return 0;
16 }
```

Defines a parallel region of code  
All code inside the {} executes in parallel



# OpenMP Hello World

```
1 #include <omp.h>
2 #include <stdio.h>
3
4 int main ()
5 {
6 #pragma omp parallel
7 {
8 // ID of the thread in the current team
9 int thread_id = omp_get_thread_num();
10 // Number of threads in the current team
11 int nthreads = omp_get_num_threads();
12
13 printf("I'm thread %d out of %d threads.\n", thread_id, nthreads);
14 }
15 return 0;
16 }
```

gcc -fopenmp hello.c -o hello

(Note: the default gcc on macosx is busted, do a "brew install gcc-15", then use gcc-15)

Defines a parallel region of code  
All code inside the {} executes in parallel



# OpenMP Hello World

```
1 #include <omp.h>
2 #include <stdio.h>
3
4 int main ()
5 {
6 #pragma omp parallel
7 {
8 // ID of the thread in the current team
9 int thread_id = omp_get_thread_num();
10 // Number of threads in the team
11 int nthreads = omp_get_num_threads();
12
13 printf("I'm thread %d out of %d threads\n", thread_id, nthreads);
14 }
15 return 0;
16 }
```

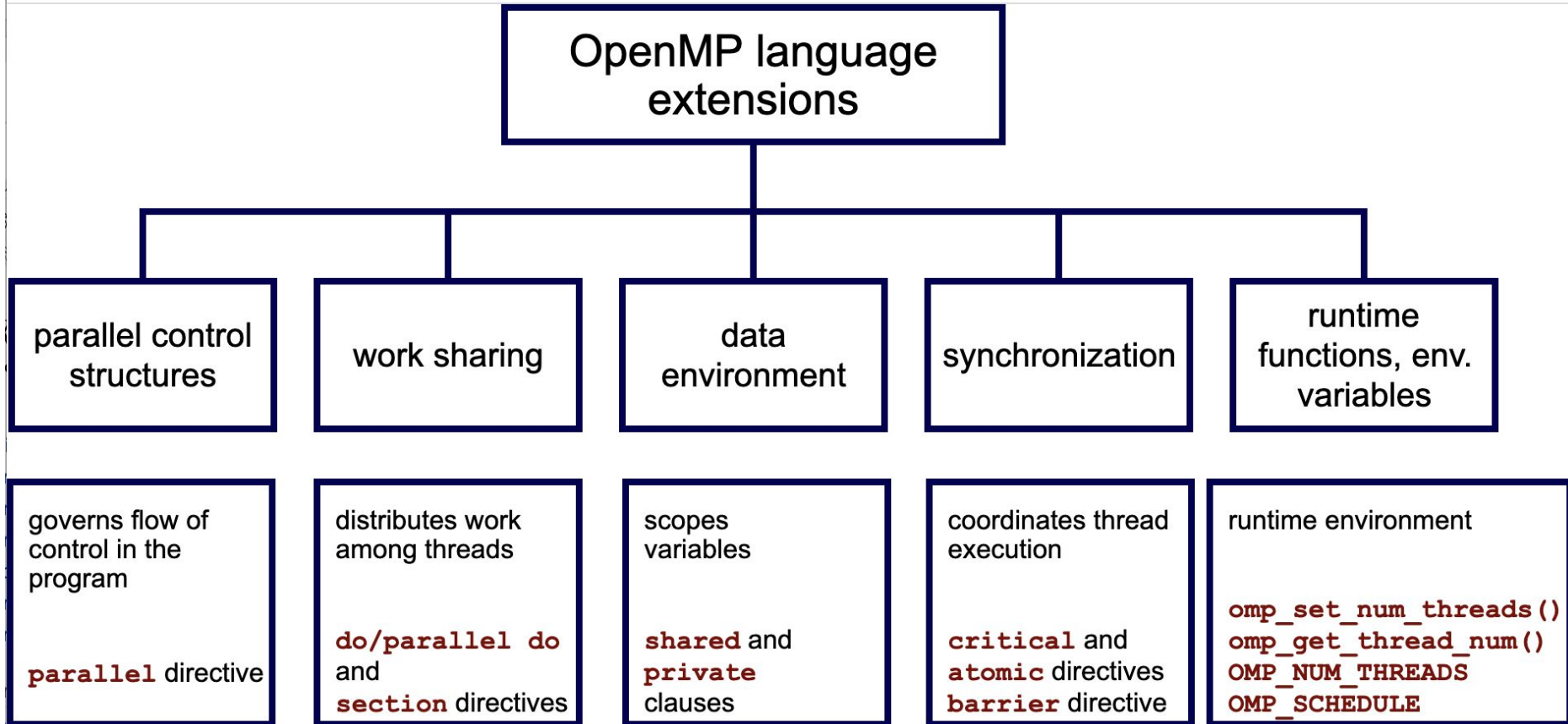
gcc -fopenmp hello.c -o hello

(Note: the default gcc on macosx is busted, do a "brew install gcc-15", then use gcc-15)

Defines a parallel region of code  
All code inside the {} executes in parallel

```
46 [wes/omptest] % gcc-10 -fopenmp hello.c -o hello
47 [wes/omptest] % ./hello
I'm thread 1 out of 8 threads.
I'm thread 2 out of 8 threads.
I'm thread 5 out of 8 threads.
I'm thread 3 out of 8 threads.
I'm thread 6 out of 8 threads.
I'm thread 0 out of 8 threads.
I'm thread 4 out of 8 threads.
I'm thread 7 out of 8 threads.
```

# OpenMP - lots of depth



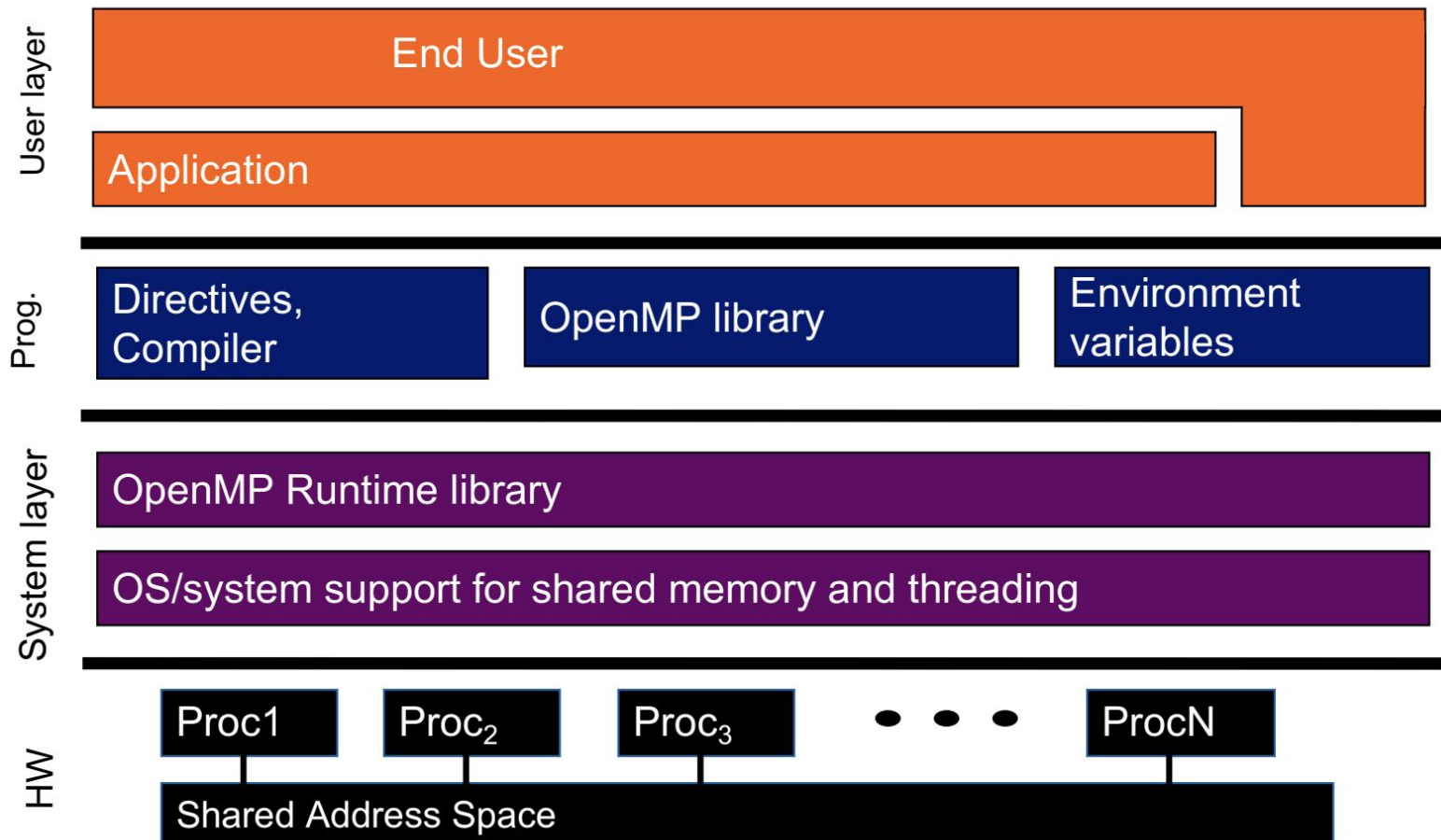


# The OpenMP Common Core: Most OpenMP programs only use these 19 items

| OpenMP pragma, function, or clause                       | Concepts                                                                                                                           |
|----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| #pragma omp parallel                                     | Parallel region, teams of threads, structured block, interleaved execution across threads                                          |
| int omp_get_thread_num()<br>int omp_get_num_threads()    | Create threads with a parallel region and split up the work using the number of threads and thread ID                              |
| double omp_get_wtime()                                   | Speedup and Amdahl's law.<br>False Sharing and other performance issues                                                            |
| setenv OMP_NUM_THREADS N                                 | Internal control variables. Setting the default number of threads with an environment variable                                     |
| #pragma omp barrier<br>#pragma omp critical              | Synchronization and race conditions. Revisit interleaved execution.                                                                |
| #pragma omp for<br>#pragma omp parallel for              | Worksharing, parallel loops, loop carried dependencies                                                                             |
| reduction(op:list)                                       | Reductions of values across a team of threads                                                                                      |
| schedule(dynamic [,chunk])<br>schedule (static [,chunk]) | Loop schedules, loop overheads and load balance                                                                                    |
| private(list), firstprivate(list), shared(list)          | Data environment                                                                                                                   |
| nowait                                                   | Disabling implied barriers on workshare constructs, the high cost of barriers, and the flush concept (but not the flush directive) |
| #pragma omp single                                       | Workshare with a single thread                                                                                                     |
| #pragma omp task<br>#pragma omp taskwait                 | Tasks including the data environment for tasks.                                                                                    |



# OpenMP basic definitions: Basic Solution stack





# CP#3

## Shared-memory Parallel (OpenMP) Vector-Matrix Multiplication

# Vector-matrix multiplication: $y = y + A * x$

Assume:

$x, y$  are vectors of length  $n$

$A$  is an  $n \times n$  matrix

for  $i=1:n$  // 1-based indexing

for  $j=1:n$

$$y[i] = y[i] + A[i,j] * x[j]$$

How many memory references?

$2n$  : we will access each  $Y[i]$  twice (r+w)

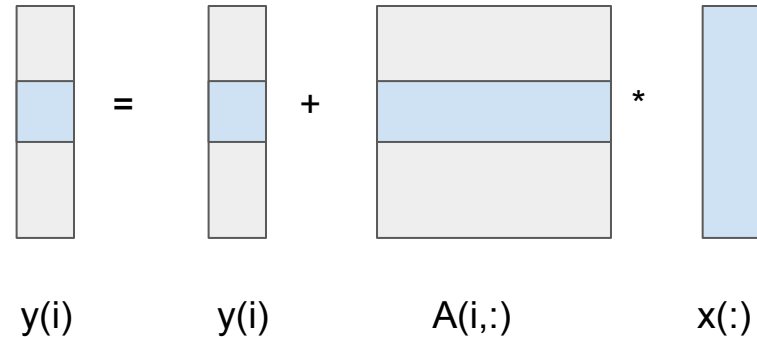
$n^2$  : we will access all of  $A$  once (r only)

$n^2$  : we will access all of  $X$   $n$  times (r only)

$m$  = number of (slow) memory refs =  $2n + 2n^2$

$f$  = number of arithmetic operations =  $2n^2$

CI =  $f/m \approx n^2/(n+n^2) < 1.0$  (not so good)



# Vector-matrix mult improvement: copy optimization

Assume:

x, y are vectors of length n

A is an nxn matrix

Note: you are not doing the block+copy optimization in VMM in CP#3

{read/copy x[1:n] into fast memory}  
{read/copy y[1:n] into fast memory}

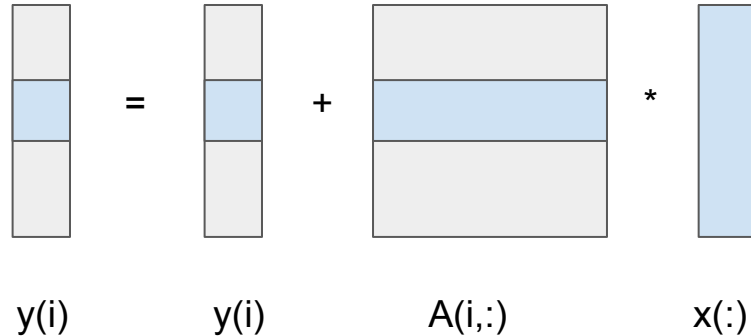
for i=1:n

{copy row i of A into fast memory}

for j=1:n

$$y[i] = y[i] + A[i,j] * x[j]$$

{write/copy y[1:n] back to slow memory}



# CP#3 – Parallel VMM – due Mon 5 Oct 2026

Parallelization of vector-matrix multiply using OpenMP

Evaluate performance at varying problem sizes and levels of concurrency

Compare to a reference implementation (CBLAS)

Versions:

- Serial VMM
- Vectorized Serial VMM
- OpenMP Parallel VMM

Deliverables:

- Code tarball
- Report

Important dates:

- 24 Sep 2026: submissions open
- 05 Oct 2026 23:59 PDT: submissions due
- 07 Oct 2026 23:59 PDT: submissions close
- 06 Oct 2026: 2 in-class presentations

# Term Project: In-class elevator pitch

## Tues 29 Sep 2026

# Semester Project Schedule

| Due            | What                                                                  | Grade %                |
|----------------|-----------------------------------------------------------------------|------------------------|
| 29 Sep 2026    | In-class pre-pre-proposal pitch session                               | 1.0% of semester grade |
| 02 Oct 2026    | Project preproposal: problem statement, approach, anticipated results | 1.5% of semester grade |
| 12 Oct 2026    | Project proposal: problem statement, approach, anticipated results    | 2.5% of semester grade |
| 01-10 Dec 2026 | Project presentations                                                 | 5% of semester grade   |
| 11 Dec 2026    | Project deadline: code+report                                         | 25% of semester grade  |

# CSC 746 Speaking Assignment Policy

## TLDR:

- There are multiple speaking assignments in this course
- They are required, not optional
- If you skip a speaking assignment, e.g., coding project presentation or term project presentation, you receive an F in the course
- Only excused absences (doctor's note, etc.) will be considered for rescheduling

# Project Preproposal Idea Pitch Session Tue 29 Sep 2026

We will do this in class on Tue 29 Sep 2026

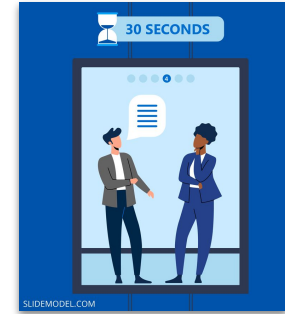
Each person gives a pitch for a project idea or two, you will receive feedback

- No slides
- Just you speaking, ok to use the whiteboard

Desired outcome:

- Everyone gets a better sense of what makes for a good project idea

Turn in: PDF with a problem statement, approach, how you might evaluate your method, potential results



- Hi, I'm Joe! (smile!)
- Problem being solved/studied
- Approach (what are you going to implement?)
- Evaluation strategy (how will you evaluate your implementation?)
- Potential outcomes (what do you hope to learn?)



# Project Preproposals – Due Friday 02 Oct 2026

1 page of info:

- What problem or question are you studying?
- How will you go about studying the problem?
  - What's the approach?
  - What are you going to implement?
- How will you evaluate the work?
- What are anticipated outcomes?

Can't decide on one? I'll review up to 2 ideas and give you feedback.

# Project Ideas

## Performance analysis and optimization of a code

- Blocking/tiling, loop reordering
- Metrics: runtime, MFLOP/s, % mem b/w, latency
- Metrics: hardware performance counters: cache miss rate, etc.

## Parallelization of a code (shared memory, distributed memory)

- Make it run faster
- Make it accommodate a problem too large for a node (memory)

# Project Planning: Technology Pathways within Reach

Take a serial code, and make it run in parallel:

- On both CPU+GPU: OpenMP
- On just GPU: CUDA
- Across multiple compute nodes: MPI (Message Passing Interface)

Instrumenting code to hardware performance counters (cache misses, etc.)

Types of applications:

- Scientific computing, esp those based on dense linear algebra solvers
- Other apps where you have a clear serial→parallel story and “code in hand”
- AI/ML: working with Python-based libraries (Torch, etc.) can be tricky but may be doable

# The End



A Warner Bros.  
PICTURE

